

AsymptoticAnalysis after the fifth review wave

A differential technical audit

Correctness at option boundaries, numerical metadata,
exact-sign recovery, and graded-tail scheduling

Repository: VladimirReshetnikov/Asymptotic

Reviewed revision: 8cee870994f506b501bae3ea6bd4a3a7edb895c1

Review date: 10 September 2026

Executed native kernel: Wolfram Language 15.0.0, Linux x86-64

Mathics source target: Mathics3 10.0.1; no Mathics interpreter run in this review

Executive assessment. The strongest new defect is small in source size but serious in kind: the recently repaired coefficient-option overload can return a successful mathematical association whose exponent contains the literal string "**Power**". A second native reproduction concerns incorrect units and sign semantics in numerical-check metadata, not an incorrect root. The Mathics numerical fallback has a separate exact-rational underflow gap. Finally, the repaired flat-product tail selector can avoid a quadratic temporary candidate list without weakening its grade-based mathematics.

This report does not reopen the old option-precedence, large-offset cancellation, or lost-exponential-grade findings. It examines the implementations introduced to address them. Two narrow fixes were exercised in the pinned standalone package on Wolfram 15: ten option checks and eight numerical-metadata checks passed. An additional 45 unmodified-package inverse comparisons passed. Separately, 39 Python test methods checked exact algebra, numerical primitives, scheduling models, and synthetic patch fixtures. These are different evidence populations, not one package acceptance total.

The archive supplies this article, a non-applying candidate-patch emitter, native and Mathics probe files, independent reference algorithms, and explicit evidence records. No complete local repository checkout, Mathics acceptance, full upstream suite, or end-to-end performance benchmark is claimed.

Contents

1	Scope, exclusions, and evidence	3
1.1	What this review adds	3
1.2	How earlier findings were excluded	3
1.3	What was actually executed	4
2	Relevant architecture and the audited boundaries	4
2.1	Core algebra versus representation adapters	4
2.2	Three concrete boundary paths	5
2.3	The Mathics adaptation boundary	5
3	N1: option-name matching can turn into string-valued mathematics	5
3.1	The precise defect	5
3.2	A public native reproduction	6
3.3	An exact oracle independent of option handling	6
3.4	Why this is a new delta rather than C09 again	7
3.5	A narrow repair	7
3.6	Executed repair checks	8
3.7	Mathics-specific consequences and limits	8
4	N2: an exact-sign test is lost to machine underflow	8
4.1	The newly added recovery path	8
4.2	A source-level witness with an executed host primitive	9
4.3	What is proved, and what still needs execution	9
4.4	Minimal exact-rational repair	10
4.5	Acceptance boundary and development opportunity	10
5	N3: numerical fields describe different quantities	10
5.1	The local solver is not the problem	10
5.2	Two native witnesses	11
5.3	Why the mismatch matters to callers	11
5.4	A compatible metadata repair	11
5.5	Focused lifecycle tests	12
6	P1: preserve grading without storing every candidate	12
6.1	The current mathematical repair should be retained	12
6.2	Where the temporary list comes from	13
6.3	The algebra of the selected bound	13
6.4	The first omitted coefficient must still be combined exactly	14
6.5	Why a naive streaming fold can change failure behavior	14
6.6	A comparison-preserving two-pass design	14
6.7	Evidence and a bounded integration plan	15
7	Additional native validation of the ordinary inverse core	15
7.1	A 45-case unmodified-package matrix	15
7.2	What the passing matrix does and does not establish	16
8	Recommended changes and acceptance criteria	16
8.1	Repair request extraction before the algebra	16
8.2	Keep exact eligibility decisions exact	16
8.3	Expose aligned local observables without breaking existing fields	17
8.4	Optimize only the new tail collection	17
8.5	Capabilities deliberately left unchanged	17
9	Artifacts, reproduction, and interpretation of counts	17
9.1	Archive contents	17
9.2	Evidence populations must remain separate	18
9.3	Final assessment	18
A	Pinned source map	19
	References	19

1 Scope, exclusions, and evidence

1.1 What this review adds

The package is maintained under the name `AsymptoticAnalysis`; the repository retains the shorter `Asymptotic` name. The reviewed revision is fixed throughout this article. The generated root `AsymptoticAnalysis.wl` was loaded directly from its commit-pinned raw URL for successful Wolfram experiments. Source inspection used the corresponding modular files. This avoids treating a moving branch, an older article, or a generated file from another revision as the tested program. [1, 3]

The requested exclusion constraint is substantial. At this snapshot, the maintained review archive has five waves, 35 retained packages, and ten retired packages represented by tombstones. The implementation register counts 231 attributed report entries from waves 1–4; that is neither a count of distinct live bugs nor a count including wave 5. [4, 3, 9]

The review therefore concentrates on four deltas, summarized in Table 1. Priority is an integration recommendation, not a numerical severity score. The option defect merits first attention because successful output can carry an invalid mathematical payload. The numerical metadata defect should also be repaired promptly, but its observed calculations were correct. The Mathics item needs a focused evaluator run before being described as an end-to-end package failure. The performance item is an implementation opportunity with an exact structural cost, not a measured slowdown.

Item	New issue or opportunity	Evidence and priority
N1	Coefficient option selection mixes name-equivalent matching with literal replacement; " <code>Power</code> " enters arithmetic.	Public Wolfram reproduction; ten candidate checks passed. First correctness repair.
N2	Mathics logarithm recovery converts an exact positive rational to machine zero before testing its sign.	Source plus executed host primitive and mathematical oracle; Mathics public path unrun. Focused compatibility repair.
N3	Local numerical fields have different observable units, while their label says otherwise.	Two public Wolfram reproductions; eight candidate checks passed. Metadata/API repair.
P1	Graded flat-tail reduction materializes mostly dominated quadratic candidate storage.	Source-derived exact count and reference-algorithm checks. Optimize after preserving comparison and cancellation contracts.

Table 1: The report’s four contributions. N1–N3 are distinct from the older defects that motivated the surrounding implementations.

1.2 How earlier findings were excluded

All five maintained wave indexes, the root retirement map, relevant implementation-register entries, and selected original review material were examined. In particular, the comparison includes C09, C21, C23, the wave-3 option-boundary stream, the wave-4 Mathics numerical work, and the wave-5 index. The indexes distinguish supplied proposals from native observations and subsequently implemented repairs; this report retains that distinction. [4, 5, 6, 7, 8, 9, 3]

This was not a word-for-word rereading of every appendix of all 35 retained articles. Repository code search returned no useful full-text matches, so an exhaustive novelty claim would be unjustified. Instead, the selected findings are tied to narrow code changes that postdate their nearest earlier reports. The included `evidence/novelty-ledger.md` records the boundary explicitly.

For N1, the old problem was that stored observable power defeated an explicit override. The current

code attempts to honor that override; the new problem is the incompatible equivalence relations used in doing so. For N2, earlier numerical work and report 45’s rational *root-seed* proposal are not this issue: the new guard concerns the sign of a *logarithm factor*. For N3, large source-offset cancellation is not alleged to remain broken; the new local-coordinate report misdescribes its own fields. For P1, exponential grading is already repaired; the new concern is how the repaired implementation schedules and stores candidate bounds. [3, 7, 9, 1, 11, 12, 13]

Known subjects such as complex-modulus rewriting, general remainder regularity, native backend defaults, certificate branch identity, broad resource taxonomy, and higher-grade remainder export are not repackaged as new findings here. Their omission is deliberate, not an assertion that every related obligation is closed.

1.3 What was actually executed

The successful connector returned the kernel identifier

15.0.0 for Linux x86 (64-bit) (May 6, 2026).

Several other connector calls failed, and external Python execution from that kernel was unavailable. Those failures are service/access limitations, not package failures. Subsequent successful calls loaded the pinned standalone, reproduced N1 and N3, and exercised the candidate changes described below.

The independent local environment was Python 3.13.5, SymPy 1.14.0, and mpmath 1.3.0. It did not contain Mathics. The exact Python operation used by the inspected Mathics rational conversion was tested there, but this does not amount to running the complete Mathics evaluation chain. Local network access did not yield a complete checkout. Consequently, the patch-emitter CLI was tested with synthetic Git fixtures; its literal replacements were separately matched against and, where applicable, executed in the real remote standalone.

Evidence files are human-readable records and transcriptions, not claims of cryptographically authenticated sessions. The native observations include an excluded harness attempt: an incorrect **AssociateTo** call failed to insert eight intended checks. Its apparent all-passed flag covered only the ten already-present checks. Clean, separate ten-check and eight-check reruns are the acceptance evidence used here. This matters because a summary Boolean is meaningful only after checking the recorded population.

2 Relevant architecture and the audited boundaries

2.1 Core algebra versus representation adapters

The ordinary engine represents a precision-tracked power-log jet by a triple

$$J = (T, P, D), \quad T = \{(\alpha_i, Q_i(\ell))\}, \quad \ell = \log u,$$

interpreted as

$$\sum_i u^{\alpha_i} Q_i(\log u) + \mathcal{O}\left(u^P (1 + |\log u|)^D\right), \quad u \rightarrow 0^+.$$

The source maintains sparse ordered weights, polynomial logarithmic coefficients, and precision transport. The inverse constructor adds a model, source and target charts, an observable power, complete coefficient blocks, and provenance. The public result is an association wrapped by **GeneralizedSeries**. [1, 10]

These layers have different obligations. A coefficient model may legitimately support algebraic manipulation of a symbolic observable parameter, whereas a numerical checker needs an actual numerical observable. An ordinary retained coefficient is not itself an interval certificate. A finite expression and its source coordinate need not have the same units once a power observable is requested. Much of this report concerns information crossing those boundaries, rather than an error in Lagrange inversion itself.

2.2 Three concrete boundary paths

N1 sits between a public option sequence and the coefficient generator. The result-object overload chooses an observable power, converts its convention at an infinite endpoint, then delegates to the model-association overload. That overload feeds the chosen value into `lagrangeCoefficient`. Using the wrong selection semantics can therefore contaminate both the coefficient and its exponent without disturbing the sparse engine. [1]

N2 sits between an exact expression and a numerical recovery heuristic. The Mathics-only late adapter rewrites selected package-owned numerical consumers to use `mathicsNumericalN`. Its fallback splits certain positive products inside logarithms. A machine approximation appears in a *guard*, before the requested arbitrary-precision evaluation is retried. [11]

N3 sits between the local numerical solve and the returned report. The improved solver works in a local source displacement and reconstructs the absolute source value for presentation. A powered observable uses the signed displacement, however, and the old explanatory field describes neither that sign nor the distinction between a root and its observable correctly. [12]

The flat-sector path is separate. It stores coefficient jets in exponential sectors, with independent inner power-log errors and an omitted sector tail. P1 concerns the temporary collection used to choose the dominant omitted sector, not the ordinary jet representation or a proposed unification of all scales. [13]

2.3 The Mathics adaptation boundary

The package's Mathics support is not merely a different executable running otherwise untouched definitions. The inspected source includes a private compatibility context and late rewrites of selected package definitions. These adapt evaluation features, association operations, numerical calls, and other primitives without intending to replace the user's global `System`` definitions. [14, 11]

That arrangement is valuable for isolation, but a successful Wolfram test does not exercise the Mathics branch. N1's candidate uses an option mechanism implemented by both evaluators, yet the actual Mathics `OptionValue` and `FilterRules` implementations are not identical to Wolfram's. N2's repair is in a conditional module that a Wolfram load never executes. Claims of parity must therefore be attached to specific paths and tests, not inferred from a successful load or from the existence of a built-in name. [19, 14]

3 N1: option-name matching can turn into string-valued mathematics

3.1 The precise defect

The current object overload contains the following selection logic:

```
Module[{explicit = FilterRules[{opts}, "Power"], power},
  power = If[explicit == {}, a["Power"],
    "Power" /. explicit];
  (* delegate using the selected power *)
]
```

This code implements two different notions of equality. Wolfram's `FilterRules` treats a string key pattern as matching a symbol with the corresponding name. Literal replacement of the expression `"Power"` does not turn that symbol rule into a string rule. A rule can therefore be selected as relevant, yet fail to replace the expression used to obtain its value. [1, 16]

The result is not simply an ignored option. Since the selected list is nonempty, the default branch is not used. The literal string survives as the value of `power`, and is passed into the mathematical coefficient calculation.

3.2 A public native reproduction

After loading the pinned standalone in Wolfram 15, construct

```
s = AsymptoticInverse[x + x^2, {x, 0}, {y, 4}];
InverseExpansionCoefficient[s, {1}, Power -> 2]
```

The returned object is an `Association`, not a failure. Its relevant fields are

```
"Weight"          -> 1
"Exponent"        -> 1 + "Power"
"Coefficient"      -> -"Power"
"UniformizerExponent" -> 1 + "Power"
```

The literal quotes are significant: this is not a symbolic parameter named `Power`. It is a string in an arithmetic expression. With `"Power" -> 2`, the same query returns exponent 3 and coefficient -2 , as expected. These observations are transcribed in `evidence/native-observations.json`.

A mixed spelling exposes another manifestation:

```
InverseExpansionCoefficient[s, {1},
  Power -> 2, "Power" -> 3]
```

The native result has exponent 4 and coefficient -3 . The first name-equivalent option should determine the value under the usual option convention; instead, the later literal string rule is the first rule able to replace the string expression. Wolfram documents that option lookup treats symbols and corresponding strings interchangeably, ignores symbol contexts for option names, and uses the first applicable setting. [17]

Option sequence	Observed pair	Interpretation
<code>"Power" -> 2</code>	$(3, -2)$	Correct control.
<code>Power -> 2</code>	String-valued	Wrong successful payload.
<code>Power -> 2, "Power" -> 3</code>	$(4, -3)$	Later literal key defeats first option.
<code>{"Power" -> 2}</code>	$(3, -2)$	Nested-list control works in Wolfram.
<code>"Power" :> 2</code>	$(3, -2)$	Delayed-rule control works in Wolfram.

Table 2: Unmodified-package Wolfram observations. Each pair is (Exponent, Coefficient). Neither nested input nor a delayed string rule is reported as a Wolfram bug.

3.3 An exact oracle independent of option handling

Let $g(y)$ be the inverse tending to zero of $x + x^2$. Then

$$g(y) = \frac{\sqrt{1+4y}-1}{2} = y - y^2 + 2y^3 - 5y^4 + \dots$$

Writing $g(y)^r = y^r U(y)^r$, where $U(0) = 1$, gives

$$U(y)^r = 1 - ry + \frac{r(r+3)}{2}y^2 + \dots$$

Thus the first correction for $r = 2$ is $-2y^3$. This conclusion does not depend on native series formatting, a package remainder convention, or the public coefficient API.

For $k \geq 1$, an exact polynomial-in- r expression is

$$[y^k]U(y)^r = \frac{(-1)^k r}{k!} \prod_{j=k+1}^{2k-1} (r+j). \quad (1)$$

For positive integer r , Lagrange inversion gives

$$[y^{r+k}]g(y)^r = \frac{r}{r+k} [t^k](1+t)^{-(r+k)}.$$

Cancelling the factor $r+k$ gives (1). For fixed k , the coefficient of $U(y)^r = \exp(r \log U(y))$ is a polynomial in r of degree at most k . Equality for all positive integers therefore extends the polynomial identity to arbitrary r . In particular, the product form avoids a removable singularity in the uncanceled formula when $r+k=0$.

The independent Python suite compares 56 coefficients, including negative powers, against a SymPy expansion of the explicit square-root inverse. It also checks the first two coefficients symbolically. These are algebraic checks of the oracle, not execution of the Wolfram package. Their role is to make the expected -2 and the surrounding coefficient conventions unambiguous.

3.4 Why this is a new delta rather than C09 again

The current C09 register entry says that explicit coefficient power takes precedence over the stored observable power and that the finite and infinite conventions have focused tests. Those existing tests use the literal string option form. [3, 15]

The present witness reaches the newly installed selection logic and produces a different failure mode: name-equivalent matching followed by literal replacement creates a value of the wrong type. It is not another observation that a conflicting string override is ignored; that control succeeds. Nor does this report propose reopening the complete wave-3 native-dispatch option design. It repairs this one coefficient consumer while retaining C09's adopted policy. [7, 1]

3.5 A narrow repair

Replace the manual filter-and-replace pair with option lookup, using the stored observable only as the omitted-option default:

```
Module[{power},
  power = OptionValue[
    {"Power" :> a["Power"]}, Flatten[{opts}], "Power"];
  (* retain the existing endpoint conversion and delegation *)
]
```

The rule-list default form is supported by `OptionValue`. Flattening the option-list structure handles nesting without traversing through the heads of rule values. The selected delayed value is resolved by the option mechanism rather than by an unrelated replacement pass. The existing conversion from a source observable power to the internal uniformizer convention at infinity remains unchanged. [17, 18, 1]

An important non-repair is to add an unconditional `NumericQ` requirement to the coefficient-model overload. The recurrence is polynomial in the observable parameter and can reasonably serve symbolic coefficient calculations. Preventing an option-key string from escaping the request layer does not require removing that algebraic capability. A future validation policy should distinguish an unresolved request token from a deliberate symbolic mathematical parameter.

3.6 Executed repair checks

The same replacement was applied to the uniquely matched text in the complete pinned standalone, saved as a temporary Wolfram file, and loaded. Ten checks passed: literal string, symbol, both mixed-key orders, nested options, omitted stored power, infinite-endpoint conversion, another symbol context, one evaluation of the selected delayed value, and no evaluation of a shadowed delayed value.

The other-context control used `AuditOptions'Power -> 2`. The finite controls compare exponent and coefficient with $(3, -2)$ or $(4, -3)$ as appropriate. The infinite control compares the override against an inverse independently constructed with power two. Testing both fields matters: a correct coefficient attached to an incorrect exponent is still a wrong contribution.

This is native validation of the replacement in the standalone, not a claim that the emitted modular patch was applied and the entire repository rebuilt and accepted. The supplied patch emitter makes that remaining integration step explicit.

3.7 Mathics-specific consequences and limits

The inspected Mathics 10.0.1 `FilterRules` primitive iterates over top-level rule elements and tests only the `Rule` head in the relevant implementation. It does not reproduce Wolfram's documented delayed-rule support through that loop. The source also does not flatten nested rule lists there. [19, 16]

The public package path may perform additional evaluation before reaching this primitive, so these source observations are not labeled fresh public Mathics failures. The archive contains probes that separately report primitive behavior and coefficient-query behavior. That separation will reveal whether a caller-level transformation masks or exposes a primitive difference.

The candidate uses `OptionValue`, whose Mathics implementation has explicit-option and default-list paths, instead of relying on `FilterRules` as an option resolver. This is a well-targeted design choice supported by source inspection, but it remains a candidate on Mathics until executed there. [19]

4 N2: an exact-sign test is lost to machine underflow

4.1 The newly added recovery path

The Mathics numerical adapter documents a failure of arbitrary-precision evaluation of logarithms of positive products, including tail targets such as an `Erfc` inverse target of 10^{-20} . It first tries ordinary numerical evaluation. When that result contains `Indeterminate`, it attempts to split qualifying logarithms of products into sums and retries. [11]

The relevant helper is

```
mathicsNumericalSplitLog[factors_List] :=
  If[And @@ (TrueQ[N[#] > 0] & /@ factors),
    Total[Log /@ factors], Log[Times @@ factors]];
```

The precision of the surrounding request is absent from the guard. Each factor is converted with default numerical precision merely to determine positivity. For exact rational factors of sufficiently

small magnitude, this is unnecessary and destructive.

4.2 A source-level witness with an executed host primitive

Take an exact rational $q_M = 10^{-M}$ and consider the positive-product logarithm

$$L_M = \log(\sqrt{\pi} q_M) = \frac{1}{2} \log \pi - M \log 10. \quad (2)$$

For every positive integer M , $q_M > 0$. There is no sign ambiguity to resolve numerically. The logarithm itself remains a moderate finite number even when the product lies outside the machine exponent range.

Mathics 10.0.1's exact rational implementation uses the following conversion when no precision is supplied:

```
def round(self, d=None):
    if d is None:
        return MachineReal(float(self.value))
    else:
        return PrecisionReal(self.value.n(d))
```

Here `self.value` is a SymPy rational. [20] The local test executes that same host-language conversion, rather than simulating it with a decimal approximation. In the authoring environment,

```
float(sympy.Rational(1, 10**323)) > 0    # True
float(sympy.Rational(1, 10**324))        # 0.0
float(sympy.Rational(1, 10**400))        # 0.0
float(sympy.Rational(1, 10**1000))       # 0.0
```

Consequently, on the inspected conversion path the guard `TrueQ[N[q_M] > 0]` becomes false, and the helper leaves a perfectly valid product logarithm unsplit. A fixed increase in the requested final precision does not address a separate machine-precision guard.

The independent suite also evaluates both sides of (2) at 100 decimal digits for $M = 20, 400, 1000, 5000$. The finite results agree within the test tolerance. This checks the proposed recovery identity; it is not a Mathics result and is not an interval certificate.

4.3 What is proved, and what still needs execution

The following conclusion is supported directly: the new helper makes an exact positivity decision through a machine conversion that loses positive rational inputs. The source and executed host primitive identify that failure mechanism without needing to guess the sign of a transcendental constant.

A stronger public statement needs care. The outer recovery path runs only if its first numerical evaluation contains `Indeterminate`. A particular Mathics version may return a different failure, an unevaluated expression, or even a successful result for a selected example. Those alternatives change whether this helper is reached. This review did not run Mathics, so it does not claim that a public `Erfc` numerical check at 10^{-400} was observed to fail.

The supplied `MathicsProbes.wl` reports the exact sign, machine value, guard result, split-helper expression, first `N` result, recovered `N` result, and split oracle separately. A real run can then distinguish failed eligibility from failed evaluation and from successful recovery. Reporting only a final Boolean would erase the distinction that this investigation needs.

4.4 Minimal exact-rational repair

The candidate preserves the existing behavior for nonrational factors but never rounds an exact integer or rational merely to test its sign:

```
mathicsNumericalSplitLog[factors_List] :=
  If[And @@ (
    If[IntegerQ[#] || Head[#] === Rational,
      TrueQ[# > 0], TrueQ[N[#] > 0]] & /@ factors),
    Total[Log /@ factors],
    Log[Times @@ factors]];
```

This is intentionally smaller than a new real-number proof engine. The rational sign is obtained by exact ordering. Zero and negative rationals remain rejected. The fallback keeps the existing behavior for the irrational positive factor, and the final numerical evaluation still uses the caller's requested precision. [11]

There are three tempting but unsuitable alternatives. Assigning higher precision to a machine zero cannot recover the discarded exact value. Raising the precision of every factor merely to decide the sign introduces avoidable work and still needs a defined policy for undecidable factors. Unconditionally expanding logarithms would broaden the branch contract and is not justified by this witness.

The general nonrational positivity heuristic remains what it was; the candidate does not claim to prove it universally sound. The purpose is to remove a deterministic exact-rational range defect from an otherwise bounded recovery path.

4.5 Acceptance boundary and development opportunity

A focused Mathics acceptance run should include both sides of the machine-underflow threshold, exact zero, negative rational factors, and a successful first evaluation that must remain unchanged. The selected requested precision should vary independently of the rational magnitude. A delayed or symbolic factor must not acquire a false positive result merely because the rational branch was added.

After the helper cases, run one actual numerical consumer whose first evaluation demonstrably enters recovery. Only then describe the result as restored public coverage. This repair does not change Mathics `FindRoot`'s numerical precision contract, solve the earlier root-seed extension, or establish arbitrary-precision root accuracy. Those are separate mechanisms. [11, 8, 9]

A useful bounded development direction is to keep numerical recovery eligibility in exact form when a cheap exact decision exists. For this particular path, the next measurable objective is invariance of rational-factor sign admission under replacing q by $q/10^m$. It is not necessary to undertake general symbolic positivity to satisfy that objective.

5 N3: numerical fields describe different quantities

5.1 The local solver is not the problem

The updated ordinary numerical checker substitutes an exact endpoint into the equation before solving in the local source coordinate. This addresses the earlier large-offset cancellation problem and preserves small displacements that would be lost by subtracting rounded absolute coordinates. [12, 3]

Let the chart be

$$x = x_0 + \sigma u, \quad u > 0, \quad \sigma \in \{-1, 1\}.$$

At an infinite endpoint the implementation uses zero shift and the corresponding signed source coordinate. In either case, the solved `localRoot` is u , not u^r .

For observable power $r \neq 1$, the stored expansion approximates the signed local observable $(\sigma u)^r$ under the ordinary constructor's convention. For $r = 1$, the checker first removes the source shift and orientation from the stored source expansion, so its local comparison is between two values of u . These distinctions are visible in the calculation itself. [1, 12]

The current report label nevertheless says:

`LocalRoot` and `LocalApproximation` are values of u for Power 1 and of u^{Power} otherwise.

There are two errors in that sentence. `LocalRoot` is always a root coordinate u . The powered local approximation includes *source orientation*; for a negative branch and odd power it approximates a negative quantity, not u^r .

5.2 Two native witnesses

The first witness uses the positive branch of $x^2 = y$ with a square observable:

```
a = AsymptoticInverse[x^2, {x, 0}, {y, 3}, "Power" -> 2];
InverseNumericalCheck[a, 4, WorkingPrecision -> 30]
```

The result contains `LocalRoot -> 2` and `LocalApproximation -> 4`, both displayed at 30-digit precision. The reference observable is 4. These fields cannot both be values of u^2 .

The second witness selects the negative branch and a cube observable:

```
b = AsymptoticInverse[x^2, {x, 0}, {y, 3},
  Direction -> "FromBelow", "Power" -> 3];
InverseNumericalCheck[b, 4, WorkingPrecision -> 30]
```

The result contains a source root -2 , `LocalRoot -> 2`, and `LocalApproximation -> -8`. The reference observable is -8 . Here $u^3 = 8$, so even the powered approximation is not described by the old label.

Both witnesses returned correct root and observable values and an approximate zero error. The issue is a public reporting contract, not a demonstrated numerical miscalculation. In particular, the known zero-remainder ratio state is not introduced as a new finding.

5.3 Why the mismatch matters to callers

A consumer reading only the label can compare 2 with 4 as if they were comparable approximations of the same observable, or cube `LocalRoot` without applying the negative source orientation. A notebook can consequently display a misleading discrepancy even while the package's own `Error` is correct. A machine-readable numerical report needs enough information to reproduce its comparison without reverse-engineering the source.

Renaming `LocalRoot` to mean a powered value would be a poor repair. Existing callers may already use it to reconstruct the source root. Likewise, changing the numerical calculation to fit the old prose would corrupt a correct local solve. The least disruptive remedy is to retain existing values, correct the description, and add an explicitly aligned comparison field.

5.4 A compatible metadata repair

The candidate adds

```
"LocalReferenceObservable" -> N[observed, wp],
"ObservablePower" -> power
```

where **observed** is already the internal quantity compared with the local approximation. It replaces the label with the following contract:

LocalRoot is always u . **LocalApproximation** and **LocalReferenceObservable** use u when **ObservablePower** is 1, and $(\text{SourceSide } u)^{\text{ObservablePower}}$ otherwise.

The existing **ReferenceRoot**, **ReferenceObservable**, **Approximation**, **Error**, and reconstruction fields are unchanged. The new pair of local observable fields has matching units by construction. [12]

The native candidate checks verify the source coordinate, local observable, and explicit power for each of the square and negative-cube examples. Two further checks verify that the existing error and approximation remain unchanged. All eight passed after the two uniquely matched metadata edits were applied to the pinned standalone.

5.5 Focused lifecycle tests

The next acceptance step should extend this pair of exact controls to a nonzero finite source offset, a negative power on an admitted branch, and power one on both sides. These cases test the same newly exposed field contract, not another general numerical redesign. A useful invariant is

$$\text{reported local error} = |\text{local reference observable} - \text{local approximation}|$$

at the internal comparison precision, followed by the existing output rounding. The displayed values may be rounded more coarsely than the internal operands, so equality of rounded differences should not be required digit for digit.

A second invariant reconstructs the root from **SourceOffset**, **SourceSide**, and **LocalRoot**. Large-offset cases must preserve the existing exact-substitution workflow rather than revert to subtracting rounded absolute values. This connects the new metadata to the repaired numerical method without reopening C21's original defect.

6 P1: preserve grading without storing every candidate

6.1 The current mathematical repair should be retained

For the flat-sector representation, write

$$E(u) = \exp(-c/u^p), \quad c, p > 0,$$

and consider omitted contributions bounded by

$$E(u)^k u^\alpha (1 + |\log u|)^d.$$

Exponential grade dominates every fixed algebraic and logarithmic power. A contribution of grade $k+1$ is smaller than one of grade k regardless of their fixed exponents, sufficiently close to the endpoint. The current C23 repair correctly selects the least omitted grade first and only then combines power-log bounds at that grade. [13, 3]

This report does not challenge that ordering. Nor does it propose preserving a higher omitted grade in the externally displayed remainder as a new feature; the register already records that possibility. The opportunity here is to compute the same selected bound with less temporary storage and less work on ultimately irrelevant bounds.

6.2 Where the temporary list comes from

The inspected `flatOpsMultiplyData` uses full coefficient products for retained sectors and for the first omitted sector. This allows exact cancellations in the first omitted coefficient to be recognized. Deeper omitted coefficient pairs contribute only envelopes. Those deeper candidates, input-tail times retained coefficients, and the tail product are appended to a list. `flatOpsGradedTail` subsequently filters the list to the least finite grade and combines only those entries. [13]

Let both input depths be n , all retained coefficients be nonzero, both input tails finite, and the complete first omitted coefficient nonzero. These assumptions define a structural count model; they are not a claim that every actual inverse satisfies that population.

The number of deeper coefficient pairs with indices $0 \leq i, j \leq n$ and $i + j > n + 1$ is

$$\sum_{k=n+2}^{2n} (2n - k + 1) = \frac{n(n-1)}{2}.$$

The first omitted coefficient adds one candidate, the two input-tail cross families add $2(n+1)$, and the tail product adds one. Therefore the temporary list length is

$$Q(n) = \frac{n(n-1)}{2} + 2(n+1) + 2 = \frac{n^2 + 3n + 8}{2}. \quad (3)$$

The source also has a pair-product budget check, $(n+1)^2 \leq \text{MaxTerms}$. This finding must not be described as a missing global bound: the existing preflight limits the admitted depth. The point is that a substantial quadratic temporary population remains inside that admitted work. [13]

Depth n	Tail candidate records $Q(n)$	Existing pair preflight $(n+1)^2$
1	6	4
10	69	121
30	499	961
100	5,154	10,201
140	10,014	19,881
500	125,754	251,001
1000	501,504	1,002,001

Table 3: Exact structural counts for the stated dense population. The final two rows require a raised budget. These are not wall-time or memory measurements.

At depth 140, the pair preflight still fits the default 20,000 budget, while the model creates 10,014 candidate records. Retained coefficient leaf counts and other checks can impose additional limits on a concrete example; the table does not assert that a particular depth-140 inverse is admitted.

6.3 The algebra of the selected bound

Represent a nonzero candidate by (k, α, d) . The conservative sum selector is ordered lexicographically by increasing k , then increasing α , then decreasing d . The exact zero bound is a separate neutral value, not a very small nonzero estimate.

Proposition 6.1. *For a finite collection of fixed-data candidates, retaining only those of minimum exponential grade and combining their power-log bounds yields a valid asymptotic bound for the sum of all candidates. Unknown error bounds must not be cancelled against one another.*

Proof. Let k_0 be the minimum grade. For $k > k_0$ and any fixed real A and nonnegative integer D ,

$$\exp\left(-\frac{(k - k_0)c}{u^p}\right) u^A (1 + |\log u|)^D \rightarrow 0.$$

Indeed, with $t = u^{-p}$, the logarithm of the magnitude is $-(k - k_0)ct + O(\log t) + O(\log \log t)$, which tends to $-\infty$. The finitely many later grades are therefore absorbed by any fixed selected bound at grade k_0 . Within that grade, the smaller algebraic exponent dominates; at an equal exponent the larger logarithmic degree dominates. A bound on an unknown error has no known sign, so subtraction of two bound descriptors is not evidence of cancellation. $\square$

The resulting join is associative, commutative, and idempotent at the level of conservative bound classes. Product bounds add grades, algebraic exponents, and logarithmic degrees. These facts support a compact reference implementation and strong invariants, but they do not authorize moving exact coefficient cancellation to the bound layer.

6.4 The first omitted coefficient must still be combined exactly

A degree-two example illustrates the distinction:

$$(1 + z + z^2)(1 + z - z^2) = 1 + 2z + z^2 - z^4.$$

The first omitted degree 3 cancels exactly, even though its individual pair products are nonzero. Assigning a degree-three magnitude envelope to each pair before combining coefficients would miss that cancellation and prevent the selected grade from advancing.

The current implementation intentionally multiplies and combines the complete first omitted coefficient before grading it. An optimization should preserve that behavior. It may reuse this computed coefficient; it should not recompute expensive coefficient products in a second pass merely to save a small control record. [13]

6.5 Why a naive streaming fold can change failure behavior

It is tempting to replace the candidate list by a running minimum. With ordinary rational exponents that is mathematically sufficient. The package, however, can incur symbolic proof work when comparing exact exponents. A streaming reducer may compare two candidates at a temporary minimum grade that is later dominated by a smaller grade.

For example, suppose candidates arrive at grades 9, 9, 2. The current two-stage list algorithm discovers grade 2 before comparing algebraic exponents. It never needs to compare the two grade-nine powers. A naive streaming fold may attempt that comparison immediately, consuming an expensive proof budget or producing an undecidable-order refusal for information that the final result does not need.

Thus mathematical equivalence of the final minimum is weaker than operational equivalence of the computation. Avoiding a new failure mode requires preserving which comparisons are demanded, not just their algebraic result. This point is especially relevant to exact symbolic software: throwing away a dominated result after computing it does not recover the cost or undo a refusal encountered while computing it.

6.6 A comparison-preserving two-pass design

A narrow replacement has two phases. First determine the least finite exponential grade using integer grade arithmetic, already available exact-zero information, and the combined first omitted coefficient. Then revisit only candidates at that final grade to form their power-log envelopes and reduce them. Retained-sector arithmetic remains unchanged.

```

def two_pass_tail(candidate_factory):
    least = None
    for item in candidate_factory():
        least = item.grade if least is None else min(least, item.grade)
    if least is None:
        return None # exact zero
    result = None
    for item in candidate_factory():
        if item.grade == least:
            result = join_tails(result, item)
    return result

```

The included Python reference uses rational exponents and a pure, repeatable factory. It demonstrates constant auxiliary result storage relative to the candidate stream; it does not claim constant storage for input jets, retained products, or the entire package calculation. Its simple factory can still construct candidate bound objects on both traversals. A production implementation should separate cheap grade descriptors from lazy envelope construction when that is useful.

Within the actual flat loop, the grade of a deeper coefficient pair is $i + j$, and input-tail cross grades are the recorded tail grade plus the retained sector index. This permits an integer-only grade scan or an equivalent grade-ordered schedule. The exact first omitted coefficient should be computed once and its zero status retained. If discovering whether a bound is exact zero requires actual algebra, that dependency must remain explicit; the optimization must not infer zero from an empty finite approximation with an unknown inner remainder. [13]

6.7 Evidence and a bounded integration plan

The independent tests verify the count formula, exact-zero behavior, power/log dominance, product-bound addition, associativity and commutativity, 300 deterministic random comparisons against a materialized reference, and several dense populations. A deliberately uncomparable algebraic-bound fixture confirms that the two-pass reducer skips comparisons at ultimately dominated grades, while a naive streaming fold attempts one. The first-omitted cancellation example is also checked.

No Wolfram scheduler patch or benchmark is included. The production integration should first compare selected bounds and retained jets with the current implementation under identical inputs. It should then measure candidate-record allocation separately from coefficient arithmetic, because a reduction in temporary records may not dominate total run time when symbolic simplification is expensive.

The decision criterion is concrete: preserve the exact retained expression, inner remainders, selected tail grade, selected power/log bound, and required comparison behavior, while removing the quadratic candidate collection. Any claimed speedup must be measured on both evaluators. This is a specific post-C23 optimization, not a restatement of the existing general resource-budget proposals.

7 Additional native validation of the ordinary inverse core

7.1 A 45-case unmodified-package matrix

To avoid basing the review solely on boundary failures, a bounded matrix was executed on the unmodified pinned package. It used the five forward polynomials

$$x + x^2, \quad x - x^2, \quad x + x^3, \quad x + 2x^2 - x^3, \quad 2x + x^2,$$

the three observable powers $-1, 1, 2$, and the three methods "Lagrange", "GroupedLagrange", and "Newton". For every source, a native `InverseSeries` result computed from a source series through

order eight served as the comparison oracle. The package inverse used cutoff four, and the difference of finite expressions was expanded through order three and simplified under $y > 0$.

```
oracle = Normal[InverseSeries[Series[f, {x, 0, 8}]] /. x -> y;
s = AsymptoticInverse[f, {x, 0}, {y, 4},
  "Power" -> r, Method -> method];
difference = FullSimplify[
  Normal[Series[Normal[s] - oracle^r, {y, 0, 3}], y > 0];
```

All 45 differences were zero. No case returned a failure or exceeded the three-second per-case constraint in that invocation. The native response reported `Total -> 45, Passed -> 45`, and an empty list of other results.

7.2 What the passing matrix does and does not establish

These checks support consistency of retained coefficients across three ordinary inversion methods for regular polynomial inputs, several signs, and positive and negative observables. They also provide useful controls against attributing N1 to a general error in the inversion formulas.

They do not validate analytic remainder exponents or logarithmic degrees, because the comparison applies `Normal`. They do not test flat, Fourier, logarithmic, Gamma, or Barnes inverse scales. They do not establish Mathics parity, all endpoint geometries, or a complete native superset. Both the package and the oracle run in Wolfram, so this is not an implementation-independent proof of every coefficient. The separate explicit quadratic oracle in Section 3 provides a more independent check for the principal defect witness.

The matrix should be retained as a focused regression family rather than converted into a broad coverage claim. Its axes are small and explicit enough that a failure can be localized to a source polynomial, observable power, and method.

8 Recommended changes and acceptance criteria

8.1 Repair request extraction before the algebra

Apply N1's two-line replacement in the canonical core file, preserving the current endpoint conversion. Run the ten focused option checks in both evaluators. The successful Wolfram run already establishes the narrow native mechanism; Mathics remains the necessary next integration check. Also rerun the existing coefficient-power tests rather than replacing them, because the new controls extend their syntax coverage rather than supersede their finite/infinite convention checks. [15]

The required postcondition is not merely "some association was returned." The chosen observable must agree with the first applicable name-equivalent option, the coefficient must match its mathematical oracle, and the exponent must use the same selected power. An unresolved string option token must never be accepted as a mathematical coefficient parameter.

8.2 Keep exact eligibility decisions exact

Apply N2 only to the integer/rational branch of logarithm-factor positivity. Run the primitive and helper probes before exercising a public numerical consumer. Record the first native numerical result as well as the recovered result, because recovery may not be entered on every input or future Mathics version.

The required postcondition is scale independence of exact-rational sign eligibility, not unlimited numerical success. A positive rational must not cease to qualify because its magnitude is smaller than

a machine number. Existing successful evaluations, nonpositive factors, and unsupported symbolic cases should retain their existing behavior.

8.3 Expose aligned local observables without breaking existing fields

Apply N3’s additive metadata change and label correction. Preserve the root-coordinate meaning of `LocalRoot`. Extend the paired native tests to translated charts and negative observables, checking units and orientation independently of decimal display precision.

The required postcondition is that the returned report explains the actual comparison. A caller should be able to identify the root coordinate, reconstruct the source root, and compare two local observable values with matching units. It should not need to infer whether a field has been raised to a power from the sign or magnitude of its numerical value.

8.4 Optimize only the new tail collection

Prototype P1 against the current flat-product implementation with unchanged coefficient arithmetic. Preserve first-omitted cancellation, exact-zero handling, and final-grade comparison demand. Measure record counts, coefficient work, and elapsed time separately; do not infer a speedup solely from the asymptotic storage model.

No general redesign of all series representations is required for this step. A scoped grade-descriptor traversal can coexist with the current result schema and the already-correct C23 mathematical selection.

8.5 Capabilities deliberately left unchanged

The proposed fixes do not add a new inverse family, a complex-domain calculus, a new certificate theorem, or a complete replacement for native series functions. They address small but observable gaps in newly added behavior. This is appropriate for an incremental review after five waves: improving the reliability of a coefficient query or the truthfulness of a numerical report can be more valuable than duplicating an already-recorded broad roadmap.

There is also no recommendation to discard the current exact algebra because a boundary adapter failed. The passing 45-case matrix and the exact coefficient derivation suggest the opposite for the tested ordinary cases: preserve the mathematical core and correct how requests and results cross its boundaries.

9 Artifacts, reproduction, and interpretation of counts

9.1 Archive contents

The `article/` directory contains the editable TeX source, rendered PDF, and a build script. `code/` contains the candidate emitter, the independent reference models and tests, a Wolfram reproducer, Mathics probes, and pinned Python dependency versions. `evidence/` contains the native observations, independent test results and log, scope record, and novelty ledger.

The emitter reads a clean Git checkout at the exact reviewed revision, verifies unique edit anchors, and emits a unified diff without applying it. It changes the three canonical modular source files for N1–N3. It does not include a flat-scheduling implementation, modify the user’s repository, rebuild the standalone, or certify the resulting checkout.

```
python code/independent_checks.py
python code/emit_candidate_patch.py /path/to/pinned-checkout \
  --output /path/outside/checkout/audit-candidate.patch
```

The emitter's successful Git fixture uses synthetic source context, not a locally cloned complete repository. Its pin and cleanliness checks are useful safeguards, not a filesystem freeze. Untracked files are outside its tracked-cleanliness test. All four source anchors were separately found exactly once in the real pinned standalone during native testing.

After reviewing and applying the emitted modular diff in a disposable checkout, regenerate the root distribution with the repository's builder, `python validation/build_standalone.py`, before testing the generated file. The root package itself documents that builder and explicitly identifies the canonical modular source as the editing location. [2]

The supplied Wolfram reproducer is loaded and called as follows:

```
Get["/path/to/audit/code/NativeReview.wl"];
AuditNativeReview["/path/to/checkout/AsymptoticAnalysis.wl",
                  "Baseline"]
```

Use **"Candidate"** after applying and rebuilding the proposed changes. The mode labels the run; it does not silently alter the package. Baseline runs are expected to fail some desired candidate checks while returning the observed faulty outputs. The function returns a record rather than automatically writing over the included evidence.

For Mathics, load `MathicsProbes.wl` and call `AuditMathicsProbes[packagePath]` in that evaluator. Those probes were not executed during authoring. The included WL files are reproducible equivalents of the successful connector expressions, not a claim that these complete files were themselves run as a local harness.

9.2 Evidence populations must remain separate

Population	Result	Meaning
Unmodified native inverse matrix	45/45	Retained-expression comparison on five regular polynomial sources, three powers, three methods.
N1 native candidate checks	10/10	One source replacement in a pinned standalone; option and endpoint behavior.
N3 native candidate checks	8/8	Two metadata replacements in a pinned standalone; values and units.
Independent Python methods	39/39	Algebra, control-flow models, host primitive, scheduling references, synthetic patch fixtures.
Mathics package execution	Not run	Source inspection and supplied probes only.
Full upstream suite	Not run	No release-gate or all-feature acceptance claim.

Table 4: Actual validation scope. Adding these counts into a single package acceptance total would be misleading.

Some Python methods contain multiple subcases, such as the 56 coefficient comparisons and 300 deterministic scheduling comparisons. They remain subcases of the 39 named methods, not additional package tests. The two candidate groups ran separately and do not prove that every interaction in a combined modular build has been tested.

9.3 Final assessment

The principal actionable result is N1: a supported option spelling currently produces wrong successful data, and the native-tested repair is small. N3 is an equally small but lower-severity correction to public semantics. N2 identifies a deterministic exact-rational loss inside a recent Mathics recovery

guard, with a bounded patch and an explicit remaining evaluator test. P1 offers a precise performance target while preserving the mathematical and operational safeguards of the current graded-tail repair. The review’s strongest conclusion is therefore not that the package is generally broken, nor that compatibility is complete. The tested ordinary inverse coefficients were consistent, while two boundary contracts failed publicly. Small, well-tested adapters around exact mathematics deserve the same scrutiny as the recurrence formulas: an option key, a positivity guard, or a mislabeled local observable can undermine the reliability of otherwise correct computation.

A Pinned source map

Source	Role in this review
<code>src/Kernel/AsymptoticAnalysis.wl</code>	Core jets, Lagrange coefficient recurrence, object coefficient overload near lines 1347–1370, constructor power conventions.
<code>src/Kernel/MathicsNumerical.wl</code>	Exact-integer seed adapter and numerical logarithm recovery; the split helper near lines 63–75 is the N2 focus.
<code>src/Kernel/NumericalInverseChecks.wl</code>	Local solve, signed observable comparison, and numerical-report fields.
<code>src/Kernel/FlatSectorOperations.wl</code>	Graded tail collector, omitted convolution scheduling, full first-omitted coefficient, and bound export.
<code>src/Kernel/MathicsCompatibility.wl</code>	Private adapter context and package-owned primitive binding.
<code>src/Tests/ReviewCoefficientPowerResidualLabel.wlt</code>	Existing string-option finite/infinite controls; retained, not replaced.
<code>docs/development/CODE_REVIEW_STATUS.md</code>	C09, C21, C23 and their earlier source attribution and current acceptance boundaries.
<code>external-reports/code-review/wave-*/README.md</code>	Five-wave exclusion inventory and supplied-evidence limits.
<code>mathics/builtin/options.py</code> at Mathics 10.0.1	FilterRules and OptionValue implementation differences.
<code>mathics/core/atoms/numerics.py</code> at Mathics 10.0.1	Rational.round’s default float conversion.

Function names and pinned URLs are the primary source locators; approximate line ranges are navigation aids, not a substitute for the revision.

References

- [1] AsymptoticAnalysis, canonical core source, reviewed revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel/AsymptoticAnalysis.wl>.
- [2] AsymptoticAnalysis, generated standalone and editing/build notice. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/AsymptoticAnalysis.wl>.
- [3] Code review implementation status, especially C09, C21, and C23. https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/docs/development/CODE_REVIEW_STATUS.md.
- [4] Review index and retirement register. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/external-reports/code-review/README.md>.

- [5] Retained first-wave review index. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/external-reports/code-review/wave-1/README.md>.
- [6] Retained second-wave review index. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/external-reports/code-review/wave-2/README.md>.
- [7] Retained third-wave review index, including option-boundary scope. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/external-reports/code-review/wave-3/README.md>.
- [8] Retained fourth-wave review index, including Mathics numerical evidence. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/external-reports/code-review/wave-4/README.md>.
- [9] Fifth-wave review index and overlap map. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/external-reports/code-review/wave-5/README.md>.
- [10] Ordinary series operations and representation transport. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel/SeriesOperations.wl>.
- [11] Mathics numerical adapter. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel/MathicsNumerical.wl>.
- [12] Ordinary numerical inverse checks. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel/NumericalInverseChecks.wl>.
- [13] Flat-sector operations and graded-tail reduction. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel/FlatSectorOperations.wl>.
- [14] Mathics compatibility context. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Kernel/MathicsCompatibility.wl>.
- [15] Existing coefficient-power and residual-label regression tests. <https://github.com/VladimirReshetnikov/Asymptotic/blob/8cee870994f506b501bae3ea6bd4a3a7edb895c1/src/Tests/ReviewCoefficientPowerResidualLabel.wlt>.
- [16] Wolfram Research, *FilterRules*, official language documentation, consulted 10 September 2026. <https://reference.wolfram.com/language/ref/FilterRules.html>.
- [17] Wolfram Research, *OptionValue*, official language documentation, consulted 10 September 2026. <https://reference.wolfram.com/language/ref/OptionValue.html>.
- [18] Wolfram Research, *OptionsPattern*, official language documentation, consulted 10 September 2026. <https://reference.wolfram.com/language/ref/OptionsPattern.html>.
- [19] Mathics3 10.0.1, `mathics/builtin/options.py`, particularly `FilterRules` and `OptionValue`. <https://github.com/Mathics3/mathics-core/blob/10.0.1/mathics/builtin/options.py>.
- [20] Mathics3 10.0.1, `mathics/core/atoms/numerics.py`, particularly `Rational.round`. <https://github.com/Mathics3/mathics-core/blob/10.0.1/mathics/core/atoms/numerics.py>.